



Campy AI

TECHNICAL HANDBOOK

Connecting cameras, and what happens to every frame

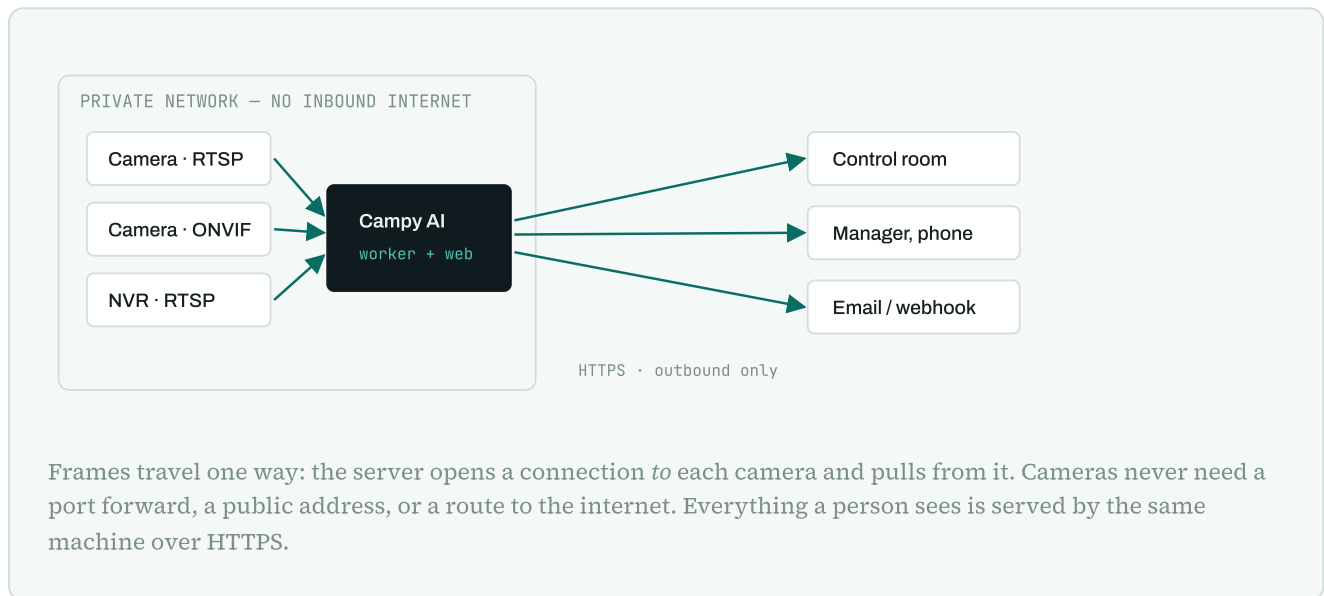
How a CCTV camera reaches the platform, how one frame becomes an alert, and how to size, secure and operate a deployment.

Technical Handbook

01	How a camera reaches the platform	Topology, protocols, and why nothing is exposed to the internet
02	Adding a camera	RTSP, ONVIF, MJPEG, HLS, file and simulated sources
03	ONVIF discovery	The SOAP calls, the digest, and the two field quirks that matter
04	Credentials and secrets	Where a password lives and where it never appears
05	The worker	Threads, pacing, reconnection and health
06	One frame, end to end	Seven stages from pixels to a stored event
07	The seven analytics	What each one computes, and what it costs
08	Events, incidents and alerts	Three layers of noise control
09	Zones	Relative coordinates, schedules and containment
10	Capacity planning	Measured numbers, and the two dials that change them
11	Training your own models	Datasets, jobs, registry and rollback
12	Operations	Running, scaling, monitoring and troubleshooting

How a camera reaches the platform

Campy AI does not replace your CCTV. It is a piece of software that sits on the same network as cameras you already own, pulls frames from them, and turns those frames into findings. No camera is modified, and no camera is exposed to the internet.



What the camera has to be able to do

Exactly one thing: emit a video stream over the network. Every CCTV camera and NVR sold in the last decade can. Campy AI reads six kinds of source:

PROTOCOL	TYPICAL SOURCE	WHAT YOU SUPPLY
RTSP	Almost every IP camera and NVR channel	Stream URL, username, password
ONVIF	Any Profile S camera — the same hardware, asked politely	IP address, port, username, password
HTTP / MJPEG	Older cameras, some door stations	Snapshot or stream URL
HLS	Restreamers, some cloud NVRs	Playlist URL
Video file	Recorded footage, for testing a site before install	Path on the server
Simulated	Built-in synthetic scenes	Nothing — used to prove the system before hardware arrives

An analogue camera is not excluded

If a site runs analogue cameras into a DVR, the DVR almost certainly exposes RTSP per channel. Point Campy AI at the DVR, one camera row per channel. The cameras themselves never change.

Decoding versus analysis

OpenCV (FFmpeg underneath) is used *only* to decode the stream into frames. Nothing about the analysis touches it: every detector, model and weight runs in Campy AI's own NumPy engine in `apps/aiengine/`. The boundary is deliberate — decoding is a solved problem with a mature library; the intelligence is the part we own.

Adding a camera

Cameras live under `/app/cameras/`. A camera belongs to a site, and a site belongs to a workspace — that chain is what keeps one customer's footage away from another's.

The fields, and what each one does

FIELD	MEANING	DEFAULT
<code>protocol</code>	Which reader to use	<code>rtsp</code>
<code>stream_url</code>	Where the video is. For ONVIF, filled in by discovery	<code>—</code>
<code>username / password</code>	Stored apart from the URL so the URL can be shown safely	<code>—</code>
<code>onvif_host / onvif_port</code>	The camera's address for ONVIF discovery	<code>— / 80</code>
<code>target_fps</code>	Frames analysed per second. The main cost dial	<code>6.0</code>
<code>source_fps</code>	What the camera emits, for reference	<code>25.0</code>
<code>rotation</code>	0/90/180/270, for a camera mounted sideways	<code>0</code>
<code>enabled_analytics</code>	Which of the seven run here	<code>[]</code>
<code>record_evidence</code>	Store an annotated JPEG with each event	<code>true</code>
<code>privacy_blur_faces</code>	Pixelate faces not matched to a consenting employee	<code>false</code>

target_fps is not the camera's frame rate

A camera streaming 25 fps with `target_fps = 6` has five of every six frames discarded without analysis. Six frames a second is ample for human behaviour — a person cannot enter and leave a restricted zone in 160 milliseconds — and it is the difference between two cameras per core and eight. Raise it only for genuinely fast events.

RTSP URLs by vendor

If you know the vendor, you can skip discovery:

```
# Hikvision – channel 1, main stream / sub stream
rtsp://user:pass@192.168.1.50:554/Streaming/Channels/101
rtsp://user:pass@192.168.1.50:554/Streaming/Channels/102

# Dahua – channel 1, subtype 0 is main, 1 is sub
rtsp://user:pass@192.168.1.50:554/cam/realmonitor?channel=1&subtype=1

# Axis
rtsp://user:pass@192.168.1.50/axis-media/media.amp

# Generic MJPEG
http://192.168.1.50/video.mjpg
```

The login goes in the login boxes, not the URL

Vendors print the address as one string —

`rtsp://admin:Tinos@1122@192.168.29.209:554/Streaming/Channels/501` — so that is what gets pasted, and the username and password boxes get filled in as well. Campy AI now splits a pasted URL apart and keeps the credentials in their own fields; before it did not, and dialling the camera sent `admin:pass@admin:pass` as the password. The camera answers **401 Unauthorized**, which looks exactly like a wrong password.

Passwords containing `@`, `:` or `/` are percent-encoded when the URL is assembled, so they no longer have to be escaped by hand.

Prefer the sub-stream

Analysis runs at 480×270 by default. Decoding a 4K main stream to throw away 97% of its pixels in the first step wastes a core per camera. A 704×576 sub-stream costs a fraction and loses nothing the detectors use. ONVIF discovery picks the sub-stream automatically; when you type a URL yourself, pick it yourself.

Testing before you commit

Test connection on the camera page opens the stream, pulls up to five frames, and reports the resolution it received — or the specific reason it failed. It is the same code path the worker uses, so a camera that passes the test will run.

ONVIF discovery

Every vendor invents a different RTSP path. ONVIF exists so you do not have to know it: give the camera its own address and credentials, and it tells you where its video is.

What happens when you press save

01 GetCapabilities, to the device service

A SOAP 1.2 POST to `http://<host>:<port>/onvif/device_service` asking for the Media category. The camera replies with the address of its media service. Firmware that rejects the category is retried without it.

02 GetProfiles, to the media service

The camera lists the streams it publishes — typically a main stream and one or two sub-streams — each with a token, an encoding and a resolution.

03 Choose the cheapest stream still worth analysing

The smallest profile at or above the analysis width wins. If every profile is smaller than that, the largest available is used instead.

04 GetStreamUri, for that profile

Asking for `RTP-Unicast` over `RTSP` returns the URL. It is written back to `stream_url` so an operator can see what was found, and so a restart need not ask again.

Authentication

ONVIF never sends the password. Each request carries a WS-Security UsernameToken whose digest is `base64(sha1(nonce + created + password))`, with the nonce and the UTC timestamp alongside it so the camera can verify without knowing what you sent. A wrong password comes back as a SOAP fault, which is surfaced verbatim — *Sender not authorized* tells an installer something that *connection failed* does not.

The two quirks that break installs

A camera reports the address it thinks it has

Behind NAT, or on a second interface, the media-service address and the RTSP URL a camera returns point at an IP you cannot reach. Campy AI rewrites the host to the one it actually connected to, keeping the port and path the camera specified, which are still correct.

The RTSP URL usually carries no credentials

ONVIF authenticates the SOAP call, not the stream. The same username and password are injected into the returned RTSP URL — unless the camera already embedded credentials, in which case its own are left alone.

When discovery fails

A failure does not raise through the worker thread. It produces a source that reports the reason, so the camera is marked offline with a message an installer can act on, visible on the camera page and in **Test connection**. If the camera has a previously discovered URL, that is used instead — a camera whose ONVIF service has crashed keeps streaming.

```
# check a camera by hand
python manage.py shell -c "\
from apps.cameras.onvif import discover_stream_url; \
print(discover_stream_url('192.168.1.50', 80, 'admin', 'pass'))"
```

ONVIF is often switched off by default

Many cameras ship with ONVIF disabled, or with a separate ONVIF user account that is not the web-interface account. If discovery reports *Sender not authorized* while the web interface accepts the same password, look for an ONVIF user list in the camera's own settings.

Credentials and secrets

A camera password is the key to a building's video. It is handled as such.

Where a password lives

- Username and password are stored in their own columns, never inside `stream_url`.
- `connection_url` assembles the two at the moment of connecting. It is never rendered into a template.
- `masked_url` is what the interface shows. Any embedded credential becomes `rtsp://...@host/path`.
- Status messages and log lines pass through a redactor, so a password cannot reach a log file by way of an error string.

Network posture

DIRECTION	TRAFFIC	REQUIRED?
Server → camera	RTSP 554, HTTP 80, ONVIF port	Yes
Operator → server	HTTPS 443	Yes
Camera → internet	Anything	No — block it
Server → internet	Outbound only: payments, email, webhooks	Optional

Put cameras on their own VLAN with no default route. Campy AI needs a route *to* them; they need no route to anything.

Video does not leave the deployment

There is no third-party AI service in the path. Every model runs in-process on the server you control. The only things that leave are the ones you configure: an email, a webhook, a payment to Razorpay or Stripe.

The worker

One worker owns one camera. It opens the stream, paces itself to the configured analysis rate, and keeps the camera's health honest.

```
# one camera
python manage.py run_camera --camera <uid>

# everything at one site, four threads
python manage.py run_camera --site <uid> --workers 4

# a whole workspace
python manage.py run_camera --organization acme-demo

# a bounded run, for a smoke test
python manage.py run_camera --all --max-frames 200
```

The loop

1. Open the source. Failure marks the camera offline with the reason.
2. Mark the camera online, recording which source served it.
3. Read a frame. Analyse it. Persist anything worth keeping.
4. Sleep the remainder of `1 / target_fps`, so a fast machine does not run ahead of the configured rate.

What happens when a camera drops

An empty read is counted, and the wait before the next attempt grows with the count — 0.5 s, then 1 s, then 1.5 s, capped at 3 s. After fifteen consecutive empty reads the camera is marked offline and the worker exits, so a supervisor can restart it cleanly rather than spinning forever.

Status follows failure count: the first two failures leave the camera **degraded**; the third and beyond make it **offline**. A camera is *healthy* only if it is online and delivered a frame within the last five minutes — which catches the case a stream stays open but stops sending.

Scaling

Each camera runs in its own thread, capped by `--workers` (default 8). Frame decoding and NumPy both release the GIL for the bulk of their work, so threads do scale — but not indefinitely. Past a handful of cameras, run several processes and shard by site, so one crash never takes every stream down at once.

```
# systemd, one unit per site
ExecStart=/opt/campyai/.venv/bin/python manage.py run_camera --site $SITE_UID --workers 4
Restart=always
RestartSec=10
```

Reports carry their evidence

Any period downloads as a PDF — one feature alone, or all seven compared — from Analytics, from a filtered Events view, or from a saved report whose format is PDF. Each finding is printed beside the frame that produced it, the detection outlined and the zone it crossed drawn on top, with camera, zone, timestamp, confidence and identity where one was recognised.

Built on ReportLab, a pure-Python wheel, because a report has to generate on the customer's own server — Windows included — without anyone installing system libraries. Evidence is best-effort: a finding whose frame has passed its retention window still appears, without a picture.

Watching a camera costs it one session

The live view does not open the camera per picture. One reader holds the stream open and publishes its newest frame; every viewer, on every screen, reads that. The camera detail page receives it as MJPEG, so the browser plays moving video rather than replacing a still every few seconds; the wall polls the same cached frame, because twenty tiles each holding a connection would pin twenty server threads.

A camera the worker is analysing is never opened twice. The worker hands its decoded frames to the same slot — so the live view shows exactly the moment the detectors are working on, and a camera that permits only one RTSP session still works while being watched. Encoding is skipped entirely when nobody is looking, and a reader closes itself 20 seconds after the last viewer leaves.

Telemetry

On exit — and continuously through the pipeline's own counters — each worker writes frames processed, events raised, errors, achieved fps, average milliseconds per frame and per-stage timings onto the camera row. That is what the camera page and the live wall read.

One frame, end to end

Seven analytics cost barely more than one, because the expensive work happens once and every analytic reads the same result.

01	Decode and normalise	I/O bound
	The frame is decoded, resized to the analysis resolution (480×270 by default) and converted to RGB. Everything downstream works at this size.	
02	Separate movement from the room	16.5 ms
	An adaptive Gaussian background model learns what the empty scene looks like, per pixel, and flags what differs. Two refinements matter: a pixel that has been foreground but motionless for a while is absorbed quickly, so a chair somebody moved stops being "a person"; and no learning at all happens inside the box of a confirmed, moving person, so somebody standing still does not dissolve into the wall.	
03	Find people and objects	25.0 ms
	Campy AI's own single-shot grid detector runs on the frame; where it is unavailable or uncertain, motion-derived candidates fill in. A boundary-gradient test rejects ghosts — a real object is edged by the image on most of its perimeter, a vacated patch of background is not.	
04	Follow them between frames	1.7 ms
	A Kalman filter per track over centre, area and aspect ratio; assignment by the Hungarian algorithm on IoU, then a second pass on proximity so a running person — whose boxes do not overlap frame to frame — stays one person rather than becoming a new one each frame.	
05	Run the analytics	17.7 ms
	Face recognition goes first, so everything after it knows <i>who</i> a track is. The rest read the same motion, detections and tracks. Each one only runs if the camera has it switched on <i>and</i> the workspace's plan includes it.	
06	Collapse repeats	negligible
	The same finding on consecutive frames is one finding. A dedupe key per condition suppresses a repeat inside a 60-frame window — ten seconds at 6 fps — before anything is written.	

Survivors become events, attach to an incident, and are offered to every alert rule.
Evidence is an annotated JPEG, optionally with unenrolled faces pixelated.

Why the order is fixed

Background subtraction, detection and tracking are the expensive stages, and they produce what every analytic needs. Running them once and sharing the result is the entire reason all seven analytics together cost about 40% more than the shared work alone, rather than seven times as much.

The seven analytics

ANALYTIC	WHAT IT COMPUTES	MS
Gesture	Posture and activity per track — walking, standing, sitting, bending, running, inactivity, loitering, falls. Speeds are normalised to body-heights per second , so one threshold holds at any camera distance and any frame rate.	0.4
Face	A 128-dimension unit embedding per face, matched against enrolled employees by cosine distance. Gated on recorded consent.	1.7
Geofencing	Point-in-polygon containment tested at a person's feet , not their centroid, plus dwell time, occupancy limits and per-zone schedules.	< 0.1
Crowd	Headcount, local density and flow coherence — the measure that separates a busy corridor from a crush risk. Falls back to density-map regression when heads occlude.	0.1
Theft	Shelf dwell, concealment gestures, carry-change and stock disappearance combined into a decaying risk score.	0.2
Objects	Watch-list classification plus geometric abandoned-object detection — a static blob that was not part of the background and has no owner nearby.	2.7
Fire & smoke	YCrCb/HSV chromatic gating, flicker variance and area growth, adjudicated by a trained classifier. Colour analysis runs at half resolution behind a cheap pre-gate.	13.6

Milliseconds are the analytic's own stage time on a busy scene at 480×270, excluding the shared stages they all depend on.

The models

MODEL	JOB	PARAMETERS
campydet	Single-shot grid detector	185k
campyface	L2-normalised identity embedding	69k
campynet	Image classifier — fire/smoke, PPE, object state	45k
campydense	Crowd density map	27k
campyseq	Temporal action model	11k

Small on purpose. A 45k-parameter classifier infers in about 0.7 ms, so the model is never the bottleneck — reading pixels is. Ten streams per core on ordinary hardware is worth more to a warehouse than one enormous backbone at 0.4 fps.

Events, incidents and alerts

An alert that fires four hundred times is worse than no alert. Three independent layers stand between a detection and somebody's phone.

Layer one — the pipeline dedupes

Inside a single camera's pipeline, the same condition is suppressed for 60 frames after it first appears. At 6 fps that is ten seconds.

Layer two — the event record counts repeats

When an event is written, a matching event on the same camera within the last **180 seconds** is not duplicated: its `occurrence_count` increments and its `last_seen_at` moves. One row, a counter, no new notification.

Layer three — the alert rule decides

CONTROL	EFFECT	DEFAULT
<code>min_severity</code>	Ignore anything below this level	medium
<code>min_confidence</code>	Ignore anything the detector was unsure of	0.5
<code>cooldown_seconds</code>	After firing, stay silent this long	300
<code>threshold_count</code>	Require this many matches before firing at all	1
<code>threshold_window_seconds</code>	...within this window	300
<code>active_schedule</code>	Only during these days and hours — e.g. after-hours only	always
<code>escalate_after_seconds</code>	Unacknowledged this long → escalation channels	off

Rules are scoped by analytic, event type, site, camera and zone, and evaluated in priority order. Delivery goes to email, webhook, Slack, Telegram, SMS or in-app. A delivery failure is caught and logged — it never stops analysis.

Incidents

Related events group into one incident, so a person crossing three zones is one thing to review rather than three. Incidents carry the status an operator actually works: open, acknowledged, resolved, false alarm.

A false alarm is training data

Marking an event a false alarm does more than close it. That frame can be pushed into a dataset as a hard negative and trained out, so the same mistake stops recurring. This is the loop that makes the system quieter over time instead of noisier.

Zones

A zone is a polygon drawn over the camera image, stored in relative coordinates so it survives a change of stream resolution.

KIND	MEANING
Restricted	Nobody should be here — or only the listed roles
Hazard	Dangerous: machinery, chemicals, a loading edge
Monitored	Watch it, but presence alone is not an event
Counting	Occupancy and flow through this area
Shelf	Stock area, watched by theft detection
Exclusion	Ignore this area entirely — a doorway to a public street, a TV screen

Details that decide whether a geofence is usable

Coordinates are relative

Stored as `[[x, y], ...]` in the range 0–1. Swap the camera for a higher-resolution model, or switch from main to sub-stream, and the zone still covers the same part of the room.

Containment tests the feet

A person is "inside" when the bottom-centre of their box is inside the polygon. Testing the centroid would count someone leaning over a line as having crossed it — the single most common source of false geofence alerts.

Zones carry their own schedule

A machine bay can be restricted between 18:00 and 07:00 and ordinary during a shift, without a second zone or a second rule.

Dwell before alert

`min_dwell_seconds` of 0 alerts on entry. Set it to 5 and somebody walking past the edge of a zone is not an incident.

Occupancy limits

`max_occupancy` turns a counting zone into a crowd rule: alert when more than N people are inside.

Capacity planning

Do not size a server from a number in a document — including this one.
Measure the machine you will deploy on.

```
python manage.py benchmark
python manage.py benchmark --width 320 --height 180
python manage.py benchmark --analytics gesture,geofence,fire
```

It runs the real pipeline over scenes of rising difficulty and prints milliseconds per frame, frames per second per core, and cameras per core at your configured analysis rate.

Measured: one core of a 2.1 GHz Xeon, all seven analytics, 480×270

SCENE	MS/FRAME	FPS/CORE	CAMERAS/CORE @ 6 FPS
Empty room	50	20	3.3
One person walking	50	20	3.3
Six people moving	73	14	2.3
Fourteen people	64	16	2.6

Plan for the busy case

Two to three cameras per core with everything switched on. A site that is quiet at 03:00 is not the case that matters; the case that matters is the shift change, when the corridor is full and every analytic has work to do.

The two dials that actually change the answer

Analysis resolution. Cost is roughly quadratic. At 320×180 the same seven analytics run in 22 ms on a quiet camera and 36 ms on a busy one — about five to seven cameras per core. Set `CAMPY_WORKER_FRAME_WIDTH` and `CAMPY_WORKER_FRAME_HEIGHT`. Below roughly 320 across, small or distant people stop being detectable at all.

Which analytics are on. Fire and smoke is the most expensive single analytic at 13.6 ms. Geofencing, crowd and theft are nearly free once tracking has run. A camera with gesture, geofencing and fire costs 41 ms on a quiet scene and 61 ms

on a busy one. A camera watching a corridor does not need theft detection; a camera over a stockroom does not need crowd analysis.

A worked example

Forty cameras, all seven analytics, 6 fps, busy site: $40 \div 2.3 \approx 18$ cores. Reduce to 320×180 and it is about 9 cores. Give half the cameras only the two analytics they need and it drops again. Storage is dominated by evidence snapshots — budget from `SNAPSHOT_RETENTION_DAYS` (30 by default) multiplied by your event rate, not by video, since Campy AI stores stills and not footage.

Training your own models

The AI is ours end to end — architecture, gradients, optimisers, training loop, weights format and inference runtime, all in NumPy. That is what makes in-product training honest: the weights a customer trains are a file on their own disk.

01 Build a dataset

Upload frames, harvest them from a camera, or import alerts an operator dismissed as false. The last of these is the most valuable source you have — it is labelled by the people who know the site.

02 Label in the browser

Classification, detection boxes or embedding identities, depending on the task. The dataset page says plainly when a class has too few examples to train on rather than letting you start a job that cannot work.

03 Train

Pick the architecture slot, epochs and learning schedule. The learning curve and the epoch log update live. A small fire classifier reaches useful accuracy in seconds; a face embedding takes longer.

04 Deploy or roll back

Each slot holds many versions with their scores. Deploying one retires the previous in the same transaction, so a slot is never briefly empty. Rollback is the same action in reverse.

Why every gradient is verified

A wrong backward pass does not crash. It quietly trains a worse model, and you find out months later from an accuracy number nobody can explain. So each layer's derivative is checked against float64 central differences, with the framework's working precision switched for the test — a float32 step of 1e-6 cannot represent the difference being measured, which makes the naive version of this check fail on correct code.

```
python manage.py train_base_models          # train and register the shipped models
pytest apps/aiengine/tests/test_gradients.py # prove the derivatives
```

Operations

Troubleshooting, by symptom

SYMPTOM	MOST LIKELY CAUSE	WHAT TO DO
Test connection fails immediately	Wrong URL, wrong port, or no route	Ping the camera from the server. Open the URL in VLC from the same machine.
<i>Sender not authorized</i>	ONVIF is disabled, or uses a separate account	Check the camera's own ONVIF user list.
<i>401 Unauthorized</i> in the log	A wrong password — or a login supplied twice	Put the credentials in the username and password fields and leave them out of the URL. Test connection now names the reason: rejected login, wrong channel, or no route.
Connects, then no frames	Stream opened but the camera is not sending	Try the sub-stream. Some cameras allow only two concurrent sessions.
Camera flaps online/offline	Network loss, or too many concurrent RTSP sessions	Reduce concurrent viewers; check switch and cabling.
Achieved fps far below target	Not enough CPU for the number of cameras	Run <code>benchmark</code> ; lower resolution, disable unneeded analytics, or add cores.
Everything is an event	Thresholds too low, or a zone covering a public area	Raise <code>min_dwell_seconds</code> , add an exclusion zone, raise <code>min_confidence</code> .
Ghost people that never move	Background still settling after a scene change	Wait for the model to re-learn; it absorbs settled pixels quickly by design.
Nothing is detected at all	Analysis resolution too low, or camera too far	People need roughly 40 px of height. Raise resolution or move the camera.

What to monitor

- `/healthz` for liveness.
- Cameras not *healthy* — online, but no frame in five minutes.

- Achieved fps against `target_fps` per camera. A persistent gap means the box is oversubscribed.
- Events per camera per hour. A sudden rise is usually a scene change, not a crime wave.
- Disk. Evidence snapshots dominate; retention is 30 days by default.

Deployment checklist

- PostgreSQL, not SQLite, for anything beyond evaluation.
- Redis and Celery if you want notification delivery off the frame path.
- Cameras on their own VLAN, no default route.
- HTTPS in front of the web tier; the worker needs no inbound port.
- One worker process per site, supervised, with `Restart=always` .
- `python manage.py benchmark` on the real hardware before committing to a camera count.